

Report

Lab0

Unfortunately, I was not able to deliver the lab before the deadline, because since I have a part-time job I could not watch recordings immediately after being published. Furthermore, in that particular case, I could not even following any class for the first two weeks of the semester due to some pre-scheduled business. Nonetheless, I wanted to join my colleagues and tried to upload my witty citation, even if I knew I would have been rightfully excluded from the review's assignment for that specific lab.

Lab1

As a first implementation I exploited a basic but effective greedy approach to the Set Cover Problem. While it does not guarantee the absolute optimal solution, it efficiently finds a reasonable approximation using iterative improvement. Integrating the tips kindly provided by Giorgio Bongiovanni and Mario Leonardo in their reviews, I started exploring a strategy for escaping local minima, implementing adaptive mutation and combining that with other techniques (in my case Simulated Annealing). That resulted to a significant decrease in terms of cost, with the cost reduction getting bigger as the number of sets and the universe size increased.

Lab2

I implemented a heuristic approach to solving the Traveling Salesman Problem (TSP) using geodesic distance and the nearest neighbor algorithm. This method approximates a solution by iteratively selecting the closest unvisited city until all cities have been visited, returning to the starting point at the end. While this approach is simple, efficient, and effective for small datasets, it does not necessarily yield the optimal solution, as it is a greedy heuristic that can get trapped in suboptimal routes. Following Sergio Lampidecchia's review and advice, I improved my approach by incorporating evolutionary algorithms, specifically a genetic algorithm with simulated annealing. This method balances exploration (searching a broader solution space) and exploitation (refining promising solutions), using operations like crossover and mutation to iteratively improve the path. By integrating this more advanced technique, I was able to obtain shorter and more optimal routes, significantly reducing the total travel distance compared to the greedy approach.

Lab3

This puzzle solver efficiently implements the A* search algorithm using a priority queue. It features an optimized heuristic that balances Manhattan distance and misplacement penalties, leading to effective state evaluation and reduced

search space. After considering Zeurehma's review, I decided to increase the number of randomizations to test how the solution works against a wider range of initial states. To tackle my other colleague Leone Fabio's issue, I enhanced my code so that user can now test multiple levels of randomization (100, 250, 500 steps). Progress is now tracked using tqdm to indicate randomization progress I have added clear output logs to compare results across different levels of RANDOMIZE_STEPS, and also encapsulated randomization into generate_randomized_state() for better reusability.

Presentation

My friend Meelad Dashti and I stumbled upon an interesting article on Medium called "How to teach AI to play games: Deep Reinforcement Learning", by Mauro Comi. The article is super interesting because it breaks down how Reinforcement Learning and Deep Reinforcement Learning work, using the game Snake as an example. We prepared some slides, covering: - The basics of Reinforcement Learning - How Q-Learning and Deep Q-Learning work - A walkthrough of the code and results from the article - Why this is more than just about Snake – how it can be applied to real-world problems.

Unfortunately we got an email from the professor saying we could not go ahead with the presentation. Apparently, the topic was considered already covered in class, even though we felt like it wasn't discussed in detail. Maybe if we'd picked a better title for our presentation proposal, it would've highlighted how our presentation was actually offering something new. Even though we didn't get to present, working on it with Meelad was super fun and really helped us understand Deep Reinforcement Learning better. Honestly, just diving into the topic, creating the slides, and discussing the ideas was worth it. Hopefully, we'll get another chance to share something cool like this in the future!

Project

Symbolic regression is a technique that identifies mathematical expressions capable of describing a dataset without relying on predefined models. It leverages genetic programming (GP) to evolve candidate solutions through mutation and crossover, optimizing them for both accuracy and simplicity. This method is particularly valuable for scientific discovery and data-driven problem-solving, as it produces interpretable models.

The implementation utilizes genetic programming techniques that involve selection, crossover, and mutation to explore mathematical expressions efficiently. Tournament selection ensures that fitter solutions are preserved while maintaining diversity within the population. The crossover process is implemented in multiple ways: nodes can be swapped at random heights, at similar heights to retain structural integrity, or at a fixed height for controlled recombination. Mutation techniques introduce variation by modifying the entire tree, changing operators or variables at random, adjusting ephemeral random constants, or

swapping subtrees to explore alternative structures.

Extensive experiments were conducted using different parameter settings and operator configurations to optimize the symbolic regression process. The primary objective was to identify the most accurate mathematical functions while minimizing the mean squared error (MSE). Various approaches were evaluated to determine how different setups influence performance. A bottleneck was surely represented by the processing capacity of the computers at my disposal, but anyhow I managed to get decent results.